

SOVereign AI

Agentic SOV Cleansing & Intelligence System

HOW IT WORKS — STEP BY STEP, IN SIMPLE WORDS

A guide for the team and for explaining the system to the judges:
what problem we solve, what each of the four agents does, how a human
stays in control, how we use the LLM safely, and how well it works.

4

collaborating agents

99%

mapping accuracy

100%

planted errors found

0

unapproved changes

ADROSONIC BUILD hackathon · Property & Casualty insurance · October 2026

Numbers in this guide come from running the current code on the provided sample files.

Contents

1. Read me first	3
2. The problem we are solving	5
3. Our solution at a glance	7
4. Step 0 — Reading the file	9
5. Step 1 — Agent 1: Sheet Discovery	10
6. Step 2 — Agent 2: Schema Mapping	12
7. Step 3a — Agent 3: Finding data-quality problems	15
8. Step 3b — Agent 3: Recommendations and review policy	18
9. Step 4 — Human review and re-reasoning	21
10. Step 5 — Agent 4: Controlled Transformation	23
11. The web application — a tour	24
12. How we use the LLM — and keep it safe	25
13. Learning from reviewers — mapping memory	27
14. Results and how we measured them	28
15. Requirements checklist	31
16. What we found and fixed along the way	32
17. Demo script and judge questions	33
18. How to run it, and where things live	35
19. Glossary	37

CHAPTER 1

Read me first

The whole system on one page. If you only have two minutes before the judges arrive, read this chapter.

What we built, in one sentence

A web application where an underwriting analyst uploads a messy client spreadsheet (a **Statement of Values**, or SOV), and **four AI agents** find the real data, match its columns to a fixed 17-column format, check every row for mistakes and explain how to fix them. The analyst approves or rejects each suggestion, and only then is a clean, standard file produced, with a full record of every change.

The journey of a file in five steps

Step	Who	What happens	Output
1	Agent 1 — Sheet Discovery	Looks at every sheet, finds the one with the property list and the row that holds the column names.	Chosen sheet, header row, clean table
2	Agent 2 — Schema Mapping	Works out which client column means which of our 17 fields (e.g. "Bldg Repl Cost" = Building Value).	Mapping with confidence and method
3	Agent 3 — Data Quality & Reasoning	Checks every row (missing values, negative money, years in the future, wrong codes, ...) and proposes fixes in plain English.	Quality report + review queue
4	Human reviewer	Approves, rejects (with a note), edits or escalates each suggestion. A rejection makes the agent think again.	Decisions with name and time
5	Agent 4 — Transformation	Applies only the approved changes, converts types, writes the clean file and the audit log.	Cleaned_SOV.xlsx + Audit_Log

Key numbers (measured on the current code)

What	Result	Target in the brief
Column mapping accuracy on the 5 sample files	103/104 headers = 99.0%	≥ 74%
Mapping accuracy on never-seen header names (held-out test)	295/300 fields = 98.3%	≥ 74%
Planted errors detected (real sample files)	70/70 = 100%	≥ 90%
Planted errors detected (30 generated files)	385/385 = 100%	≥ 90%
Recommendations with a written rationale	100%	100%
Changes applied without human approval	0	0 (disqualifying otherwise)
5,000-row file, all four agents end to end	15.5 seconds	no timeout; < 60 s goal
Automated tests	569 passing	—

THE FIVE RULES WE NEVER BREAK

1. **Nothing changes without a human “yes”.** Agents only recommend; Agent 4 applies approved changes only.
2. **We never invent data.** Missing stays blank. Unknown values are flagged, not guessed.
3. **Rules first, AI second.** Fast, predictable checks run on every row; the LLM is only asked when rules are unsure.
4. **Every decision is explained.** Each recommendation has a reason, a confidence and an “I am unsure about...” statement.
5. **Everything is recorded.** Every applied change is in the audit log with before, after, who approved it and when.

SAY THIS TO THE JUDGES

“Our agents do the tedious work and explain themselves; the underwriter keeps the final say on every change.”

“We put deterministic rules first so detection is predictable, and use the LLM only where judgement is needed — and even then we validate its answer.”

The problem we are solving

What an SOV is, why it is messy, and exactly what the hackathon asks us to deliver.

What is a Statement of Values (SOV)?

When a company buys property insurance, it sends the insurer a spreadsheet listing every building it owns: where it is, what it is made of, how old it is, how much it is worth, and whether it has fire sprinklers. This spreadsheet is the **Statement of Values**. The insurer feeds it into risk models that decide the price of the policy. If the SOV is wrong, the price is wrong.

The trouble: every client sends a different spreadsheet. Today an exposure-management team cleans each file by hand. That is slow, error-prone and does not scale.

Why the files are hard — real examples from the sample files

Problem	Real example we met	How we handle it
Data is not on the first sheet	SOV_K4T9 has 7 sheets (Summary, Glossary, Disclaimers, two location lists...)	Agent 1 scores and ranks every sheet
Header is not on row 1	SOV_B4ID: column names are on row 12, under a title block	Agent 1 scores the first 30 rows as possible headers
Different column names	"Bldg Repl Cost New", "Insured Occupancy", "#Floor", "%Sprink"	Agent 2 uses 8 layers of evidence
Same word, different meaning	"Buildings" holds €22 million values (money), not a count	Agent 2 checks the values, not just the name
Junk rows inside the data	"Total" rows, footnotes, blank separator rows	Agent 1 removes and lists them
Placeholders	Year Built = 9999 (means "unknown") in 23 rows	Agent 3 proposes "leave blank", never a guess
Mixed formats	"\$1,200,000", "1.2M", "Texas" vs "TX", ZIP 802.0 (lost zeros of 00802)	Agent 3 standardises, with approval
Numbers that are not Y/N	"% Sprinklered" = 0.57 (partly sprinklered)	Flagged — a business decision, not ours
Non-US data	SOV_K4T9 lists sites in Portugal, Spain, Germany	US state/ZIP rules skip non-US rows

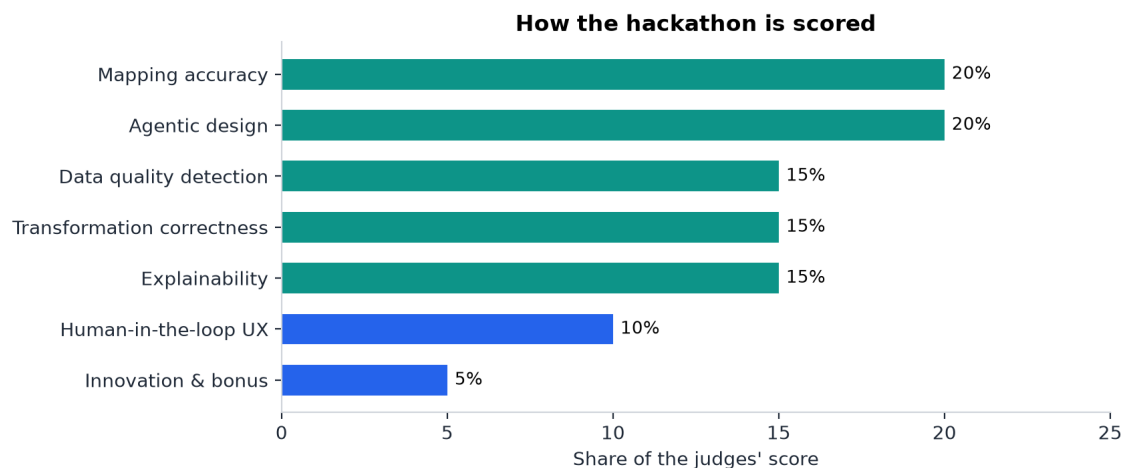
What we must deliver (the five objectives)

- **Ingest and detect structure** — accept .xlsx/.csv, find the right sheet and header row.
- **Map columns to the 17-field target schema** — exact, fuzzy and semantic matching, with a confidence score per column.
- **Detect data quality issues** — missing values, wrong types, logical errors, format inconsistencies.
- **Explain every recommendation** — plain English, with what the agent is unsure about.
- **Human approval, then transformation** — a UI to review; changes only after approval; export Cleaned_SOV.xlsx.

The target format: 17 fields

Field	Type	Field	Type
Reference	Text	Contents	Number (money)
Address	Text	BI (business income)	Number (money)
City	Text	Occupancy	Text
State	Text (2-letter)	Construction	Text
Zip	Integer	Storeys	Integer
County	Text	Number of Buildings	Integer
Country	Text	Year Built	Integer
Building Value	Number (money)	Fire Sprinklers (Y/N)	Y, N, Y13 or Y(13R)
Other	Number (money)		

How the judges score us



Agentic design and mapping accuracy are worth the most; explainability and human-in-the-loop together are another 25%.

HARD RULES (CONSTRAINTS) — breaking these is disqualifying

C-01 No change without explicit human approval. **C-02** Never invent or fill missing data. **C-03** Output has exactly the 17 columns, in order. **C-04** At least four distinct agents sharing state. **C-05** Every mapping and change explained. **C-06** Process all sample files without crashing. **C-07** Files named Cleaned_SOX.xlsx and Audit_Log.xlsx / Audit_Log.json.

CHAPTER 3

Our solution at a glance

Four agents in a line, a human in the middle, and one shared record that everyone reads and writes.

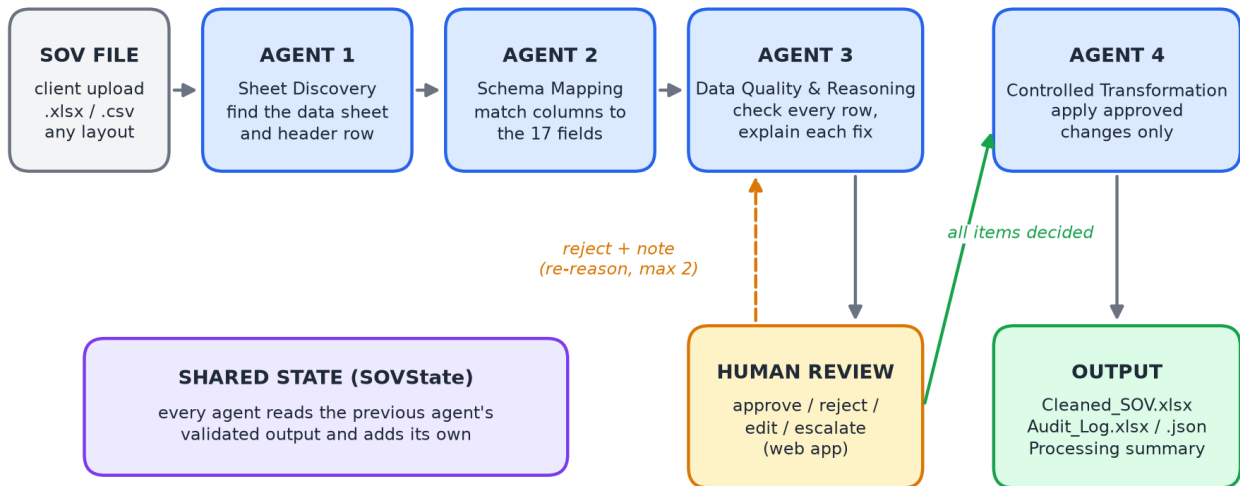


Figure 1 — The four agents, the human review gate and the re-reasoning loop. All agents share one state object.

How the agents talk to each other

The agents do not pass loose files around. They share one structured object called **SOVState** — think of it as a case folder that travels down the line. Agent 1 writes the chosen sheet and clean table into it; Agent 2 reads that table and adds the column mapping; Agent 3 reads both and adds the quality report and the review queue; the reviewer's decisions are added; Agent 4 reads everything and writes the outputs. Each agent only works on what the previous agent has already validated (requirement NFR-7).

If a reviewer changes a column mapping during review, the input to Agent 3 has changed, so the pipeline **automatically re-runs Agent 3** on the new mapping. Decisions on items that did not change are kept; new findings go back to the reviewer.

Design principles

Principle	What it means in practice	Why it matters
Deterministic first	Every row is checked by fixed rules. The same file always gives the same findings.	Predictable, testable, fast; detection does not depend on an LLM's mood.
LLM only where judgement helps	Ambiguous column names, re-thinking after a rejection, business-impact notes.	Saves cost, keeps results reproducible, limits risk.
Never trust the LLM blindly	Its answer must be from a closed list, may not invent values, and is re-checked by the validators.	Brief warns: "AI models can hallucinate mappings or invent data."
Recommend, don't modify	Agents 1-3 never change the data. A fingerprint check proves the data is untouched.	C-01: silent mutation is disqualifying.
Group, don't flood	395 identical fixes become one review item, not 395.	A reviewer can finish a queue of ~50 items, not thousands.
Explain in plain English	Every item: what is wrong, the proposed fix, a before/after example, the confidence, the doubt.	Explainability is 15% of the score; a non-technical reviewer must understand.

One file's journey: SOV_K4T9

Stage	What happened on SOV_K4T9.xlsx
Agent 1	Scanned 7 sheets. Chose "Locations" (confidence 0.89) over "Original from Client A" (0.78); 5 sheets rejected by name (Glossary, Summary...). Header found on row 6; 34 data rows.
Agent 2	Mapped 10 columns (overall confidence 0.93); 7 columns left unmapped on purpose (Latitude, TIV, PD...). "Buildings" → Building Value because its values are money.
Agent 3	Found 51 issues (intake score 62/100): 23 Year Built placeholders (9999), missing addresses, 3 unusually large Contents values. Built 28 review items.
Human	Approve All cleared the safe items; the rest were decided one by one.
Agent 4	Wrote Cleaned_SOV.xlsx (single sheet, 17 columns), Audit_Log.xlsx/json and a processing summary.

Step 0 — Reading the file

Before any agent works, the file is opened safely and turned into tables.

- **Formats:** .xlsx, .xls and .csv. Every sheet of a workbook is read.
- **No header is assumed.** Sheets are read as raw grids, because the header may be on row 12 (Agent 1 finds it).
- **CSV typing:** a CSV has no cell types, so plain numbers like 450000 are turned into numbers. Values with a leading zero (like ZIP "02108") stay text, so the zero is never lost.
- **Readable errors (NFR-4):** a corrupt file says "The file is not a valid Excel workbook. It may be corrupt or password-protected...", an empty CSV says "The CSV file is empty.", a .txt says "Unsupported file type". The app never shows a crash.
- **Files are closed after reading** (on Windows an open file would stay locked).

Step 1 — Agent 1: Sheet Discovery

Goal: find where the real property data lives before anything else happens.

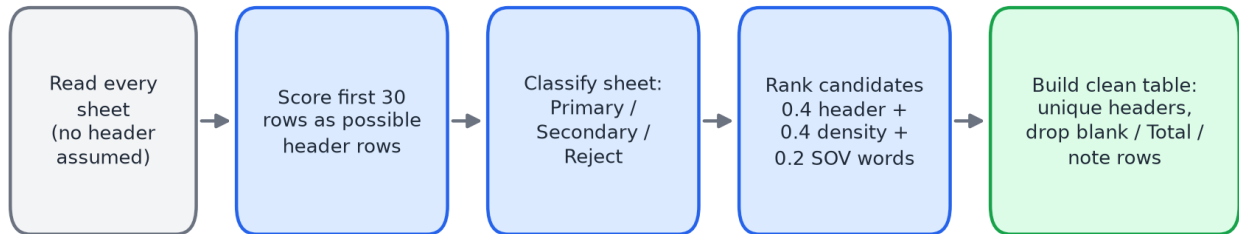


Figure 2 — Agent 1 in five steps.

1. Finding the header row

For each of the first 30 rows, Agent 1 asks: does this row look like column names? It scores signals such as: how many cells are filled, how many are text, whether the words are SOV words (address, city, zip, construction, sprinkler...), whether the values are unique, and whether the rows *below* look like data (numbers, dates). The best-scoring row wins. Rows that look like repeated data records are rejected, so a data row is never mistaken for a header.

2. Classifying each sheet

Class	Meaning	Example
Primary	Has location identity (address, location ID) and property attributes (construction, values) with enough data.	"Locations", "SOV", "23-24 Values"
Secondary	Related but specialised or partial (e.g. deleted locations, BI-only sheet).	"Deleted Locations", "2023 BI Values"
Reject	Not property data: glossary, disclaimers, summaries, vehicle lists, or no plausible header.	"Glossary", "All Autos", "Summary"

3. Ranking — choosing between candidates

Some workbooks have two sheets that both look like the main list. SOV_K4T9 has "Locations" and "Original from Client A". Agent 1 gives every candidate a **rank score**:

$$\text{rank score} = 0.4 \times \text{header score} + 0.4 \times \text{data density} + 0.2 \times \text{SOV-word score}$$

Sheet (SOV_K4T9)	Class	Confidence	Decision
Locations	Primary	0.89	Selected
Original from Client A	Primary	0.78	Not chosen
Glossary	Reject	0.90	Rejected
Vendor Disclaimers	Reject	0.90	Rejected
Confidentiality	Reject	0.90	Rejected
Summary	Reject	0.90	Rejected
YOY Comparison	Reject	0.90	Rejected

If no sheet is Primary, the best Secondary sheet is used and a warning tells the reviewer. Every decision comes with reasons ("Has location identity and property attribute columns; Header detected at row 6; Data density 68%").

4. Building a clean table

- **Unique column names:** repeated headers ("2024 - Increase 10%" three times in SOV_B4ID) become "...", "... (2)", "... (3)"; blank headers become "Unnamed_14". This used to crash the pipeline.
- **Blank, "Total" and note rows are removed** and listed for the reviewer, so a total row is never treated as a building.
- **Excel row numbers are kept**, so every finding later says "Excel row 16", exactly where the reviewer will look.
- Agent 1 changes no values — it only decides which cells form the table.

SAY THIS TO THE JUDGES

"Agent 1 doesn't just take the first sheet: it ranks every sheet with a scored, explained decision, finds headers anywhere in the first 30 rows, and removes totals and footnotes so they are never mistaken for buildings."

Step 2 — Agent 2: Schema Mapping

Goal: decide which client column means which of our 17 fields, with a confidence and a reason.

This is the hardest part of the challenge and 20% of the score. Client headers are abbreviated, misspelled, or completely different words. Agent 2 collects evidence from eight layers — cheap and certain first, expensive and uncertain last — and then makes one global decision so that no two columns claim the same field.



Cheap, certain evidence first -> expensive, uncertain evidence last

Figure 3 — The eight evidence layers of Agent 2. Numbers on the right are the typical confidence each layer gives.

The layers in simple words

Layer	How it works	Example
1. Exact / alias	Cleaned header equals a field name or an approved insurance alias. “#” is read as “number”.	“Zip” → Zip; “BPP” → Contents; “Loc #” → Reference
2. Fuzzy spelling	RapidFuzz similarity ≥ 0.75 (as the brief requires) — and the key words must really match, so “Account Name” is not taken for “County Name”.	“Constrution” → Construction
3. Concept keywords	Expands abbreviations (bldg, yr, blt, const, occ, cnty...) and recognises insurance concepts.	“Bldg TIV” → Building Value; “Yr Blt” → Year Built
4. Memory	A reviewer confirmed this header in an earlier file (see Chapter 13).	“Fire Prot.” learned once, reused
5. Value profile	Looks at the data: 4-digit years, money amounts, 5-digit ZIPs, Y/N codes, state codes.	Column of 1985, 1990... looks like Year Built
6. Embeddings + re-ranker	Two small language models compare the meaning of the header with each field description.	“Location Name” $\approx$ City (0.59 → review)
7. LLM	Only for columns still unclear. Sent in batches; sample values that could identify people or addresses are masked.	Ambiguous leftovers
8. Veto + assignment	Values that contradict a field cancel it; then a global one-to-one assignment (Hungarian algorithm).	€22M values cannot be “Number of Buildings”

Worked example: the “Buildings” column

Evidence	What it says
Header	“Buildings” is an approved alias for Number of Buildings (a count).
Values	22,756,726 · 24,854,236 · 5,880,070 — these are money amounts, not counts.
Value veto	A count field cannot have a median above 500 → “Number of Buildings” is rejected.
Disambiguation	Ambiguous word “building(s)” + money values → Building Value (confidence 0.97).
Result	Mapped to Building Value, method “value_disambiguation”, reason written in the mapping.

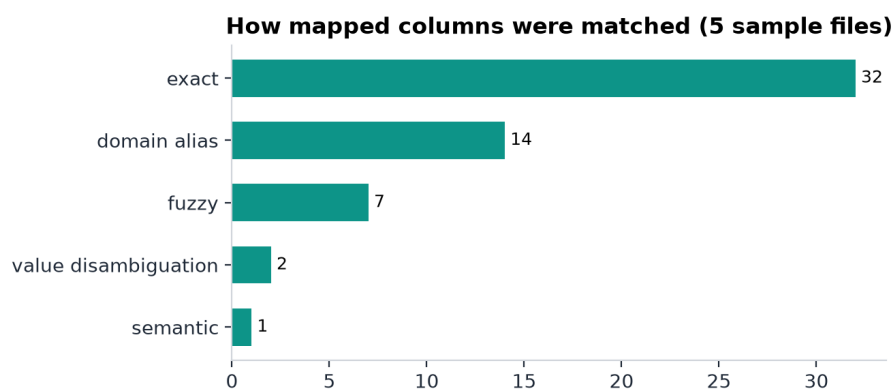
The opposite case is handled too: in SOV_Q8B3 a column called “Building” contains names like “Animal Control Center”, so it is *not* mapped to Building Value (text cannot be money).

Confidence and what happens next

Confidence	What it means	What the reviewer sees
≥ 0.90	Strong (exact, alias, clear fuzzy match)	Eligible for “Approve All”
0.50 – 0.90	Probable	Individual review item
< 0.50	Not enough evidence	Column left unmapped and flagged “human_review_required” (FR-2)

The mapping output (format required by the brief)

```
{
  "sheet_identified": "Locations",
  "header_row": 6,
  "mappings": {
    "Location ID": {"target": "Reference", "confidence": 1.0, "method": "exact"},
    "Buildings": {"target": "Building Value", "confidence": 0.97,
                  "method": "value_disambiguation"},
    "Location Name": {"target": "City", "confidence": 0.59, "method": "semantic"},
    "TIV": {"target": null, "confidence": 0.10, "method": "none",
            "flag": "human_review_required"}
  },
  "unresolved_count": 7,
  "overall_confidence": 0.93
}
```



How the mapped columns of the 5 sample files were matched.

SAY THIS TO THE JUDGES

"Our mapping is not just string matching: it combines names, spelling, insurance abbreviations, the actual values, meaning, and — only when needed — an LLM. And values can veto a name: a column of millions is never a building count."

CHAPTER 7

Step 3a — Agent 3: Finding data-quality problems

Goal: check every row with fixed rules, so detection is complete and repeatable.

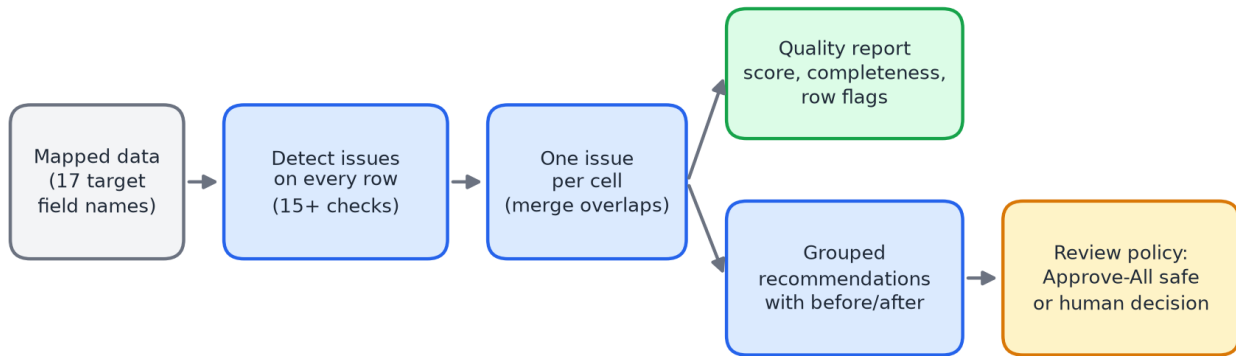


Figure 4 — Agent 3: detect, merge to one issue per cell, report, recommend, apply the review policy.

The checks

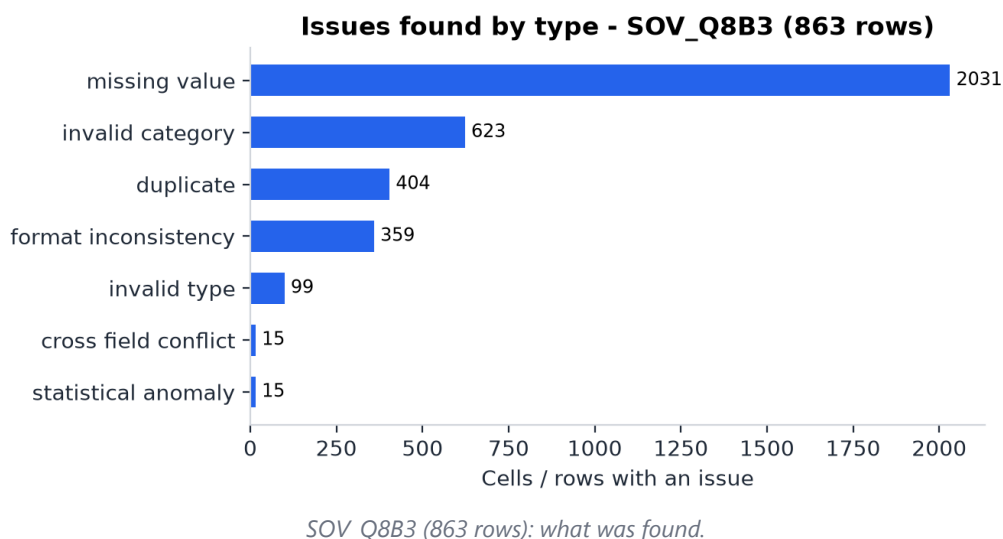
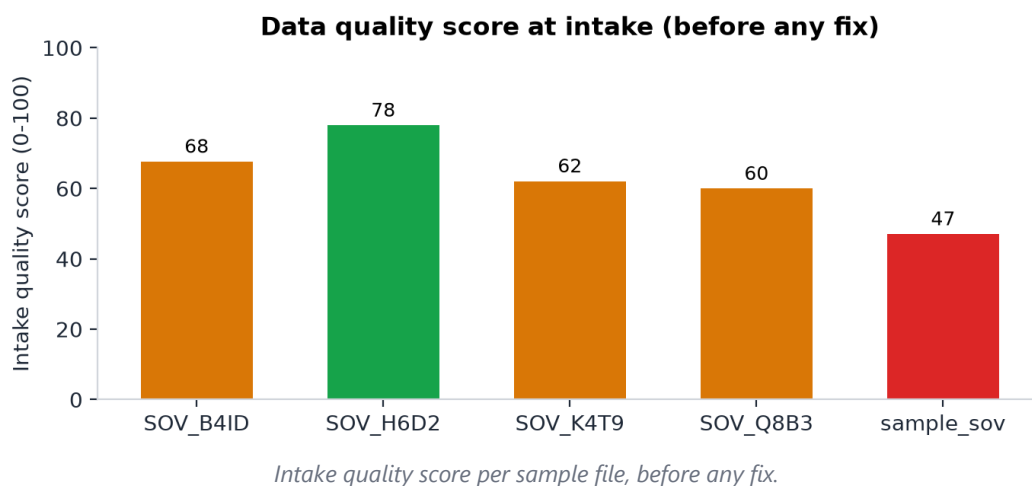
Check	Catches	Example	Severity
Missing values	Empty required cells; whole missing or empty columns	No Address in 16 rows	Medium (Low if optional)
Type	Text where a number belongs, etc.	Contents = "Included in Bldg"	High for money
Negative values	Money or counts below zero — also hidden in text	-500000, "(500)", "-\$500"	High
Range	Storeys or Number of Buildings below 1	Storeys = 0	Medium
Future year	Year Built after this year	2099; text "2099"	High
Placeholder year	Codes meaning "unknown"	9999, 0	Medium/High
Sprinkler codes	Anything not Y, N, Y13, Y(13R)	YES, No, 1, 0.57, "Manual Pull"	Medium/High
Format	Right meaning, wrong form	"\$1,200,000", "1.2M", "EUR 1.200.000,50", "Texas", extra spaces	Low
ZIP	Lost leading zeros, bad structure	802.0 → 00802; "ABCDE"	Medium
State	Unknown state; names vs codes	"XX"; "Calif." → CA	Medium / Low
Duplicates	Repeated Reference; repeated whole rows	Reference 400 appears 3×	Medium
Cross-field	Values that disagree with each other	ZIP 02108 with State CA; all insured values = 0	Medium
Statistical outliers	Unusually large or small values (log scale for money)	Contents of 289 million	Low/Medium (advisory)

Smart details that avoid false alarms

- **Country-aware:** US state and ZIP rules only run on US rows, so “Galicia” in a Spanish row is not an error.
- **Excel numbers are understood:** a ZIP stored as 75219.0 is fine (just a type conversion); only ZIPs that lost leading zeros are flagged.
- **Percentage sprinkler columns:** the agent works out if a column is 0–1 or 0–100. 0 → N, full → Y, partly sprinklered → a human decides.
- **One issue per cell:** if a year is both “in the future” and “an outlier”, the reviewer sees one issue, with the second as supporting evidence.
- **Outliers on a log scale:** insured values range from thousands to hundreds of millions; this cut false outlier flags on SOV_Q8B3 from 271 to 0 for Contents.

The quality report

Agent 3 produces an **intake quality score** (0–100): the share of cells with no problem, weighted by how serious each problem is (High = 1, Medium = 0.5, Low = 0.25). It also gives per-field **completeness** (how many cells are filled) and **validity** (how many filled cells are correct), plus a list of problem fields for every row (FR-3).



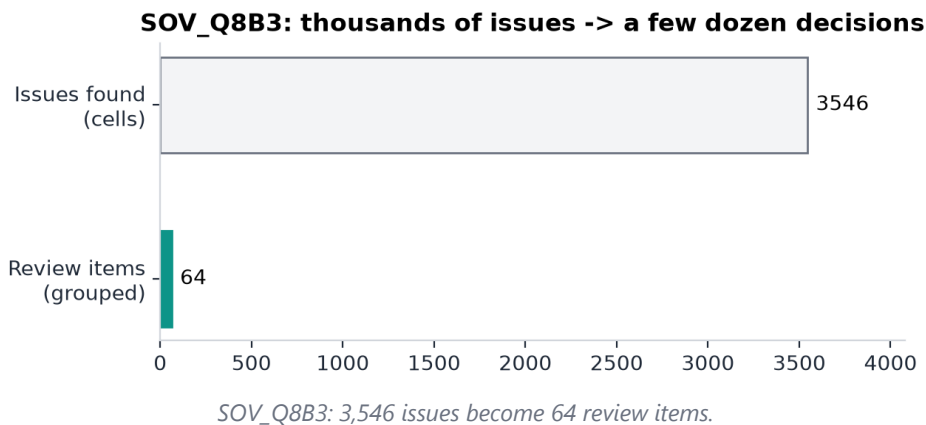
SAY THIS TO THE JUDGES

“Detection is 100% rule-based and runs on every row, so recall doesn't depend on an LLM. We also understand real spreadsheet quirks — Excel ZIPs, percentage sprinkler columns, European rows — so we don't drown the reviewer in false alarms.”

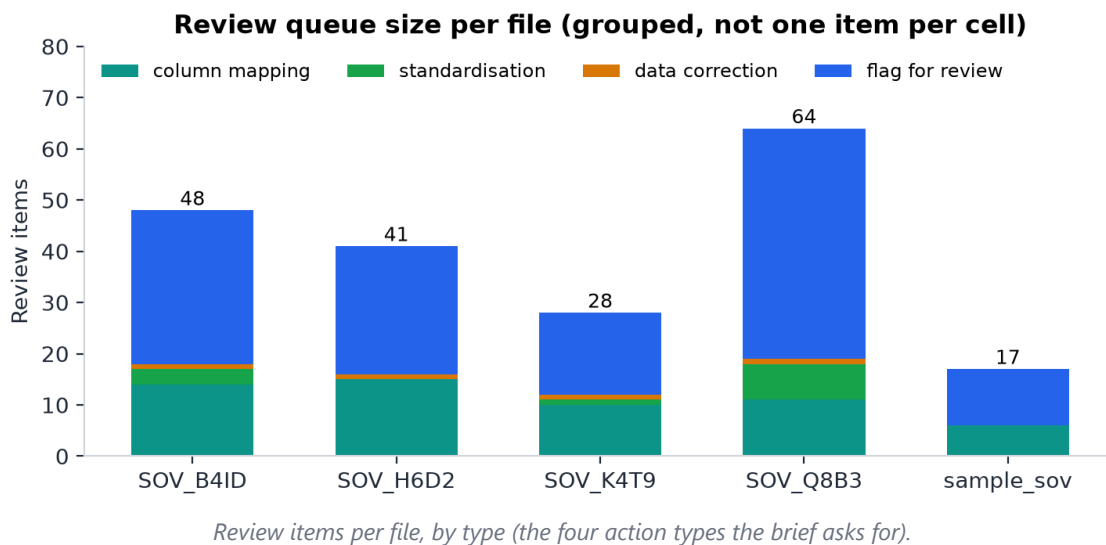
Step 3b — Agent 3: Recommendations and review policy

Goal: turn thousands of findings into a short, explained list of decisions.

Grouping: thousands of findings, a few dozen decisions



Identical problems with the identical fix are grouped. For example, on SOV_Q8B3 the 395 sprinkler cells that say "No" become a single item: "Fire Sprinklers (Y/N): change 'No' to 'N' (395 rows)".



The four action types (FR-4)

Action type	Meaning	Example
column_mapping	Accept (or change) which column feeds which field	Map "Buildings" → Building Value
standardisation	Same meaning, standard form — lossless	"No" → "N"; "Texas" → "TX"; "\$1.2M" → 1200000
data_correction	Changes a value based on an inference — always individual review	9999 → blank; ZIP 802 → 00802; sprinkler 1 → Y
flag_for_review	No safe fix exists; the value is kept and shown to a human	Negative Building Value; Storeys "2 + B"

Anatomy of a recommendation (a real one)

Part	Example (SOV_Q8B3)
Title	Fire Sprinklers (Y/N): change 'No' to 'N' (395 rows)
Action / operation	standardisation / canonicalize_sprinkler
Rationale	395 rows in Fire Sprinklers (Y/N) do not meet the rule: Value should use one of canonical sprinkler representations: Y, N, Y13, Y(13R). 'No' is a recognised synonym of the canonical sprinkler code 'N'. For example 'No' becomes 'N'. If approved, Agent 4 applies exactly this conversion; if rejected, the source values are kept and the item is re-assessed using your note.
Uncertainty	The source value is a recognised alternate representation, but normalisation requires approval before transformation.
Before → after	No → N (sample rows with Excel row numbers)
Confidence	0.95 (fix confidence) — separate from the detection confidence
Evidence	rule id, expected condition, observed values, related rules, statistics
Policy	Approve-All eligible (lossless, ≥ 0.90, Medium severity)

Never guessing: what the agent refuses to do

Situation	What the agent proposes	Why
Negative Building Value	Keep and flag	Zero or flipping the sign would change the insured exposure
Missing value	Keep blank	C-02: never invent data
Sprinkler "MAYBE" / 0.57	Keep and flag	Not a Y/N fact; a business decision
Year "1920's", "1938-1939"	Keep and flag	Several possible years
Year 9999	Propose blank (individual approval)	A known "unknown" code — blank is honest, a year would be a guess
Statistical outlier	Keep and flag	Large sites exist; unusual ≠ wrong

Review policy: what can be bulk-approved

Every item needs a human decision. The policy only decides **how**: in one click with “Approve All”, or individually.

Approve All (bulk) — all must be true	Individual decision — any of these
Standardisation or column mapping Lossless (only the form changes) Fix confidence ≥ 0.90 Severity Low or Medium Found by a rule, first attempt	Data corrections (inferences) Flags High or Critical severity Confidence < 0.90 Proposed by the LLM or re-reasoned after a rejection

SAY THIS TO THE JUDGES

“A reviewer never faces thousands of cells: identical fixes are grouped, safe format fixes can be approved in one click, and anything that changes meaning needs an individual decision with a written reason.”

Step 4 — Human review and re-reasoning

Goal: keep a person in charge, and let the agent learn from a rejection.

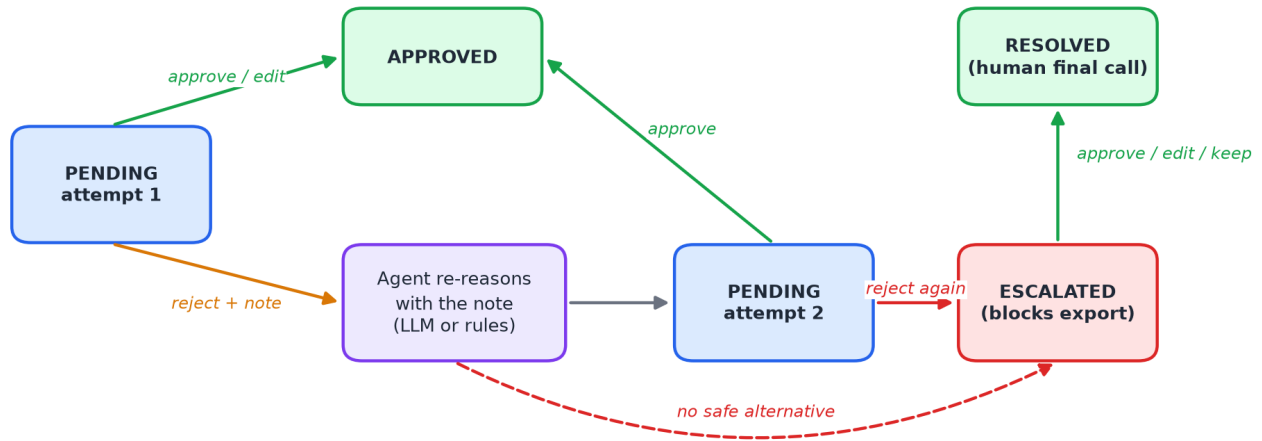


Figure 5 — Life of a review item. At most two attempts, then a human makes the final call.

The four decisions

Decision	What happens
Approve	The item is marked approved, with the reviewer's name and the time. Agent 4 will apply it.
Reject (note required)	The agent re-thinks using the note and proposes attempt 2. A second rejection escalates the item.
Edit	The reviewer types the right value (or picks another field for a mapping). The value is checked by the same rules first — an invalid edit is refused with a clear message. Rows with different source values are edited one by one.
Escalate	Sent for a senior decision; blocks export until resolved.

How re-reasoning works

- **If the LLM is on**, it receives the item, its evidence, the rejected proposal and the reviewer's note (names and addresses masked).
- **If not**, a rule-based reasoner takes over: if the note names a valid value ("these are Y(13R) systems"), it proposes it; otherwise it proposes keeping the source value.
- **Guardrails on every proposal**: the operation must be allowed for that field; it may not repeat a rejected proposal; a new value must be an allowed value or appear in the note (no invention); converted values must pass the validators again.
- If no safe alternative exists, the item is escalated straight away.

Real example

Step	What happened (SOV_H6D2, live test)
Attempt 1	Fire Sprinklers (Y/N): change 1 to 'Y' (6 rows) — the column is a 0-1 percentage, 1 = full coverage.
Reviewer	Reject: "Sprinkler % of 1 here means 1 percent coverage... client confirmed these are not sprinklered, use N".
Attempt 2 (LLM)	"Reviewer clarified that a value of 1 represents 1% coverage, indicating no sprinkler installation; therefore the correct canonical representation is 'N'." Title: "... set 'N' (6 rows) — revised after rejection".
Reviewer	Approve → Agent 4 writes N in Excel rows 16, 18, 21, 24... and records it in the audit log.

Export gate

Cleaned_SOV.xlsx can only be produced when **no item is pending or escalated** (FR-5). Until then the Export button is locked and shows how many items remain.

SAY THIS TO THE JUDGES

"Rejecting is not a dead end: the agent re-reasons with the reviewer's note, at most twice, and every proposal is checked so the LLM cannot invent a value. After two rejections a human makes the final call."

Step 5 — Agent 4: Controlled Transformation

Goal: apply exactly what was approved — nothing more — and prove it.

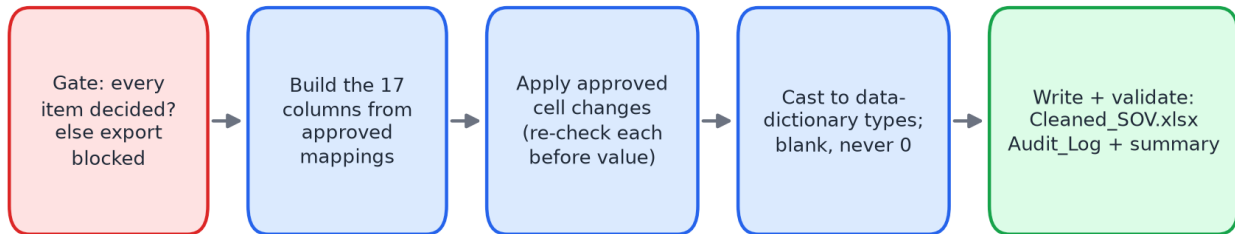


Figure 6 — Agent 4.

- **Gate:** refuses to run while anything is pending.
- **Columns:** builds the 17 columns in the required order from the approved mappings; fields with no source become empty columns.
- **Approved changes only:** each change is re-checked ("is the cell still what the reviewer saw?") before it is applied.
- **Same code as the preview:** the before/after a reviewer saw is produced by the same functions that Agent 4 applies, so the result is exactly what was approved.
- **Types:** converts to the data dictionary types (Year Built 1970.0 → 1970). Missing stays blank — never 0 or "N/A". A value that cannot be stored (e.g. "TBD" in a money column) is left blank and its original value is kept in the audit log.
- **ZIP:** stored as an integer (as the dictionary says) but displayed as 5 digits, so 00802 stays visible.
- **Self-check:** after writing, the file is validated: single sheet "Cleaned_SOV", exactly 17 headers in order, correct types, no merged cells, valid sprinkler codes.

Outputs

File	What it contains
Cleaned_SOV.xlsx	One sheet "Cleaned_SOV": headers in row 1, data from row 2, no merged cells, no colour formatting, blanks for missing values.
Audit_Log.xlsx / Audit_Log.json	One entry for every applied change — including every type conversion — with source column, target column, transformation, before, after, confidence, approved_by, timestamp, Excel row, rationale.
Processing_Summary.md / .json	Sheet chosen, mapping summary, intake score, review decisions, changes by type, timings.
sheet_manifest / schema_mapping / quality_report / review_queue / agent_trace .json	The intermediate results of every agent, for transparency.

SAY THIS TO THE JUDGES

"Agent 4 is deliberately boring: it can only apply approved changes, it re-checks each one, it logs every single changed cell, and it validates its own output against the schema."

CHAPTER 11

The web application — a tour

A Streamlit app that a non-technical reviewer can use. Start it with: `uv run streamlit run app/ui/streamlit_app.py`

Area	What you see and do
Sidebar	1 · Your name (recorded as <code>approved_by</code> — buttons stay disabled until entered) · 2 · Upload the SOV · Options (LLM on/off, LLM notes, learning memory) · 3 · Analyse file · key status ("Groq keys: 2 · all available") · Start over.
Progress bar	Always visible: "Review: 23 of 41 decided · 18 pending", then "All items decided — ready to export".
Overview tab	Headline numbers, the live agent workflow diagram (green done, amber waiting for you, red failed, dashed re-reasoning loop), and a next-steps checklist.
Sheets tab	Ranked sheet manifest with confidence and reasons; rows left out of the table.
Mapping tab	Every source column → target field with a confidence bar, method and review flag; target fields with no source; the mapping JSON.
Data quality tab	Intake score, issues by type and severity (charts), completeness and validity per field, problem fields per Excel row.
Review tab	Cards with colour badges (severity, action, status, confidence, "Approve-All eligible"), rationale, uncertainty, LLM business-impact note, before/after table, evidence and decision history. One-click Approve; Reject / Edit / Escalate open a small dialog. Filters, paging, "Approve All safe items". You stay on the Review tab after every action.
Export tab	Before and after tables side by side; Export button (locked until nothing is pending); downloads for Cleaned_SOV.xlsx, Audit_Log.xlsx, Audit_Log.json and the processing summary.

TIP FOR THE DEMO

Enter your name first — without it the decision buttons stay greyed out on purpose, because every decision is signed.

Do one test run before the judges arrive so the AI models are already loaded (first analysis takes ~15 s, later ones a few seconds).

How we use the LLM — and keep it safe

Groq-hosted openai/gpt-oss-120b, temperature 0, structured JSON answers only.

Where the LLM is used (and where it is not)

Used for	Not used for
Agent 2: columns still unclear after all rule layers (batched)	Detecting errors (100% rules)
Agent 3: re-thinking a rejected item with the reviewer's note	Choosing values freely (closed lists only)
Agent 3: short business-impact notes on items that need a decision	Changing data (never — Agent 4 applies approved changes)
	The quality score

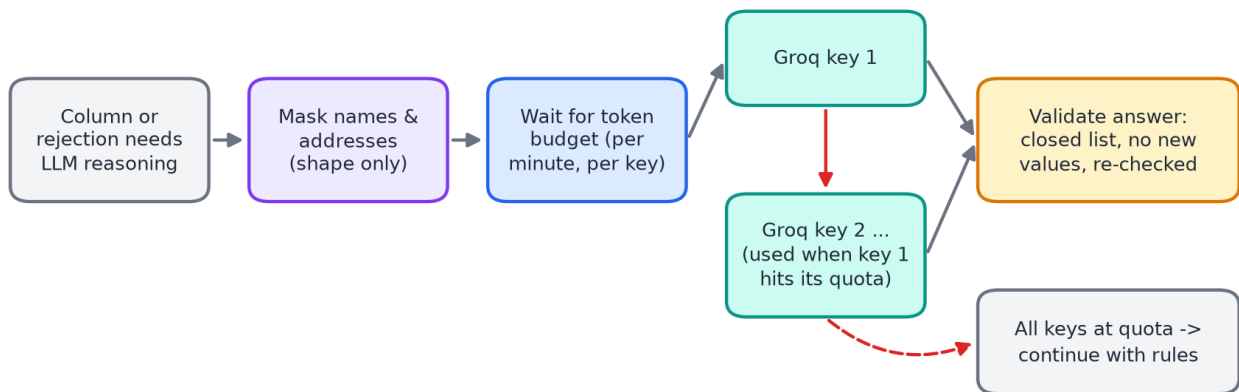


Figure 7 — Every LLM call: masked input, token budget, key failover, validated answer.

Safety measures

- **Privacy:** values that could identify a person, company or address are sent as a shape only — “505 Gentry Memorial Hwy” becomes “999 Aaaaaa Aaaaaaaa Aaa”. Short codes like “TX”, “Y”, “1985” are kept. Agent 3 masks Reference, Address and City entirely.
- **Closed answers:** the LLM must pick one of the 17 fields (or none) and one operation from a fixed list; anything else is discarded.
- **Validation:** every LLM proposal is checked by the same rules as a human edit. It cannot repeat a rejected proposal or introduce a value that is not allowed or not in the reviewer's note.
- **Graceful fallback:** if the LLM fails or is unavailable, the system continues with the rules and marks uncertain items for review — it never blocks.

Speed and reliability on the free tier

The free Groq tier allows about 8,000 tokens per minute and 200,000 per day. Three improvements made the LLM reliable:

Problem we hit	Fix
One LLM call per unclear column (17 calls for SOV_Q8B3)	Columns are sent in batches of 8 (3 calls)
Hidden “reasoning tokens” used up the budget	Low reasoning effort; realistic token estimates
Bursts caused “429 rate limit” errors	A shared token budget waits before calling instead of failing
Daily quota reached → minutes of retries	Circuit breaker: stop calling until reset, continue at once with rules (303 s → 4.5 s)
One key = one quota	Several API keys from different Groq accounts with automatic failover

```
# .env – keys from DIFFERENT Groq accounts (limits are per organisation)
GROQ_API_KEY=first-key
GROQ_API_KEY_2=second-key
GROQ_TOKENS_PER_MINUTE=7000      # per key; raise on a paid tier
```

SAY THIS TO THE JUDGES

“The LLM is an advisor, not an actor: it sees masked data, answers from closed lists, every answer is validated, and if it is unavailable the system keeps working with its rules.”

Learning from reviewers — mapping memory

Bonus objective: cross-submission learning with vector memory.

- When a file is exported, every reviewed column mapping is saved: the **header text**, its **embedding** (a numeric meaning vector) and the decision.
- Next time, Agent 2 checks memory: the same header gives strong evidence ($\approx$ alias strength); a header with similar meaning (cosine similarity ≥ 0.90) gives moderate evidence.
- **Negative memory:** if a reviewer corrected a mapping, the wrong target is blocked for that header in future files.
- **Privacy:** only headers are stored — never row values (tested). Stored in `memory/mapping_memory.json`; can be switched off.

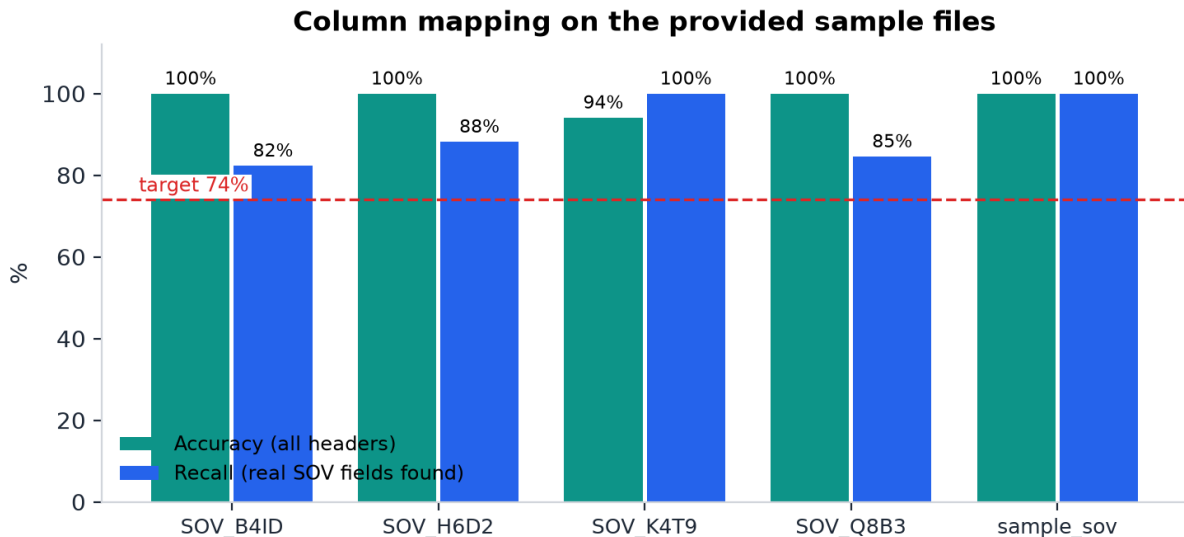
SAY THIS TO THE JUDGES

"The system gets better with every reviewed file: approved mappings are remembered, corrections are never repeated, and no client data is stored — only column headers."

Results and how we measured them

Honest numbers: what we measured, how, and what they do and do not prove.

Column mapping on the sample files



Accuracy counts every header (including correctly ignored junk columns); recall counts only real SOV fields found.

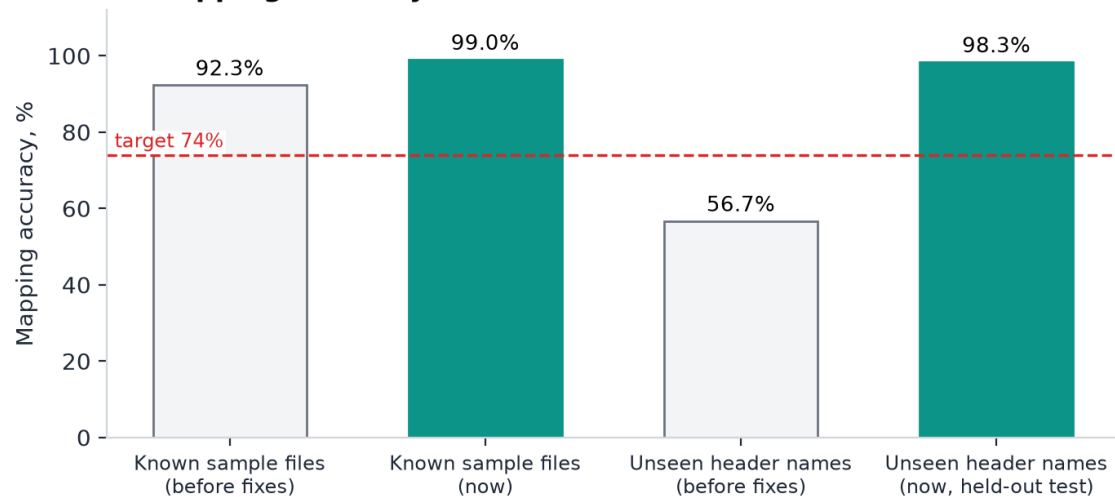
File	Correct / headers	Accuracy	Recall	Remaining miss
SOV_B4ID	27/27	100%	82%	—
SOV_H6D2	25/25	100%	88%	—
SOV_K4T9	16/17	94%	100%	'Location Name'
SOV_Q8B3	29/29	100%	85%	—
sample_sov	6/6	100%	100%	—

The one remaining miss — “Location Name” → City — is debatable (the values are place names); it is mapped with 0.59 confidence, so it goes to review rather than being silently accepted.

Held-out test: header names the system has never seen

Scores on the sample files are optimistic because we developed against them. So we built a test generator: it takes the real data, renames every header with a phrase that is **not** in our vocabulary (e.g. “Structure Value”, “Yr Blt”, “Premises Address”), moves the header row, adds title rows, Total rows, junk columns and extra sheets, and plants errors. The phrases were split in two halves: a **dev** half we were allowed to study, and a **test** half used only for reporting.

Mapping accuracy: known files vs. never-seen header names



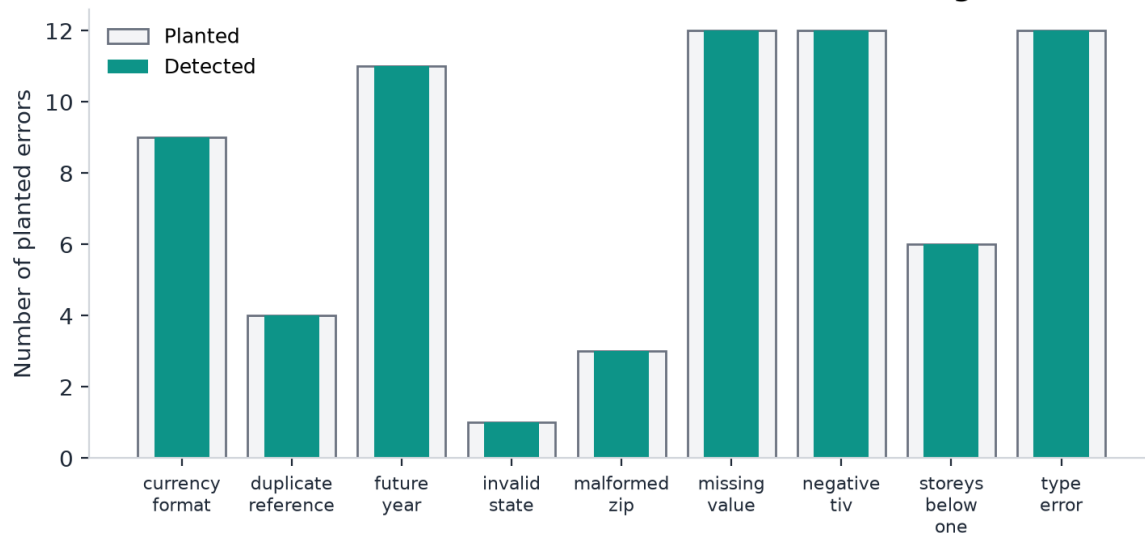
Held-out test half: 295/300 real fields mapped correctly over 30 generated files; 30/30 sheets and header rows found.

BE HONEST ABOUT THIS NUMBER

The synonym list and our abbreviation rules were written by the same people, so even the held-out score is probably optimistic. The true test is new files from the team: label them in `tests/evaluate_mapping_accuracy.py` and run it. Low-confidence columns go to review, so a miss costs review time, not wrong data.

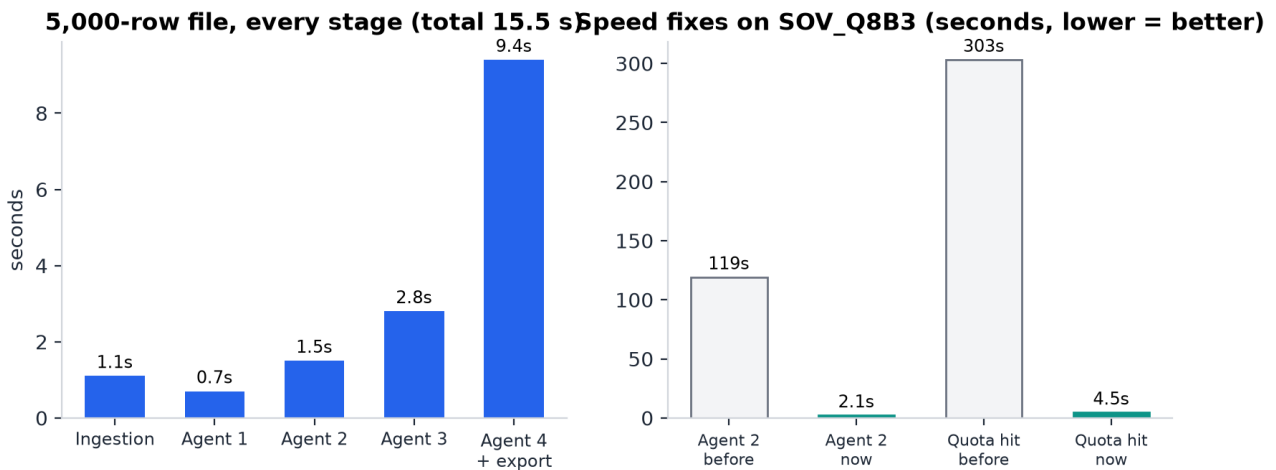
Error detection (anomaly recall)

Error detection on the real files: 70/70 found (+ 385/385 on generated files)



Every planted error type was found on the real files; the generated files add European numbers and bracket negatives.

Speed



Left: a 5,000-row file through every agent and the export. Right: two speed problems we found and fixed.

Quality of the engineering

- **569 automated tests** — unit tests for each piece, integration tests per agent, end-to-end pipeline tests, UI tests, and regression tests that run the real sample files.
- Regression tests lock in the brief's targets (mapping $\geq 74\%$, recall $\geq 90\%$) on both the sample files and the held-out variants.
- Every Agent 1–3 run proves it did not modify the data (a fingerprint of the table before and after).

Requirements checklist

Every requirement in the problem statement and where we meet it. All items were verified by running them.

ID	Requirement	How we meet it
FR-1	Upload .xlsx/.csv in a UI; multi-sheet; header anywhere; 5,000 rows	Streamlit upload; Agent 1 ranking; header scoring; 5,000 rows in 15.5 s
FR-2	Rank sheets; exact then fuzzy (≥ 0.75) then semantic; confidence + method; flag < 0.50 ; JSON	Agent 2 layers; RapidFuzz 0.75; embeddings + cross-encoder; Section 09 JSON
FR-3	Completeness per field; types; negative/future/Storeys < 1 ; formats; per-row and per-field flags	Agent 3 checks + quality report
FR-4	Recommendations with action type, before/after, rationale, confidence, uncertainty; LLM; no auto-apply; re-reasoning	Grouped recommendations; LLM notes and re-reasoning; export gate
FR-5	Show column, mapping, confidence, rationale; accept/reject/edit; before/after preview; Approve All ≥ 0.90 ; block export	Review cards and dialogs; Export tab preview; Approve All (stricter: lossless only); gate
FR-6	Approved only; exact names; types; blanks; single-sheet Cleaned_SOV.xlsx; audit log fields	Agent 4 + validation; Audit_Log.xlsx/.json
NFR-1	Mapping $\geq 74\%$	99.0% samples; 98.3% held-out (see caveat)
NFR-2	Anomaly recall $\geq 90\%$	100% (70/70 real, 385/385 generated)
NFR-3	100% of transformations audited	Every changed cell logged, incl. type casts
NFR-4	No crash; readable errors	Corrupt, empty, unsupported and missing files tested
NFR-5	17 columns, types, no merged cells	Validated after every export
NFR-6	Rationale for every decision	100% of items; sheet reasons; audit rationale
NFR-7	Shared state; validated inputs only	SOVState; Agent 3 re-validates after mapping edits
C-01...C-07	Constraints	All met (see Chapter 2)
Bonus	Vector memory; workflow visualisation; re-reasoning; intake score	All four done

What we found and fixed along the way

Running the whole system end to end on real files exposed problems that unit tests did not. These make good stories for the judges.

Area	Problem found	Fix
Agent 1	Duplicate / blank headers crashed the pipeline	Unique, cleaned column names
Agent 1	Total rows and footnotes became fake buildings	Removed and listed for the reviewer
Agent 1	Two “primary” sheets, no tie-breaker	Rank score with confidence
Agent 2	Semantic matches were silently ignored (missing score field)	Score passed through correctly
Agent 2	An LLM error erased all evidence for a column	Keep rule evidence; LLM failure is just a warning
Agent 2	“Buildings” (millions) mapped to a count	Value veto + value-based disambiguation
Agent 2	“Account Name” fuzzy-matched to County	Key words must match, not just letters
Agent 2	Every method reported as “global_assignment”	Real method reported (exact, fuzzy...)
Agent 2	Weak semantic score overrode a confident LLM answer	Stronger of the two is used
Agent 3	865 Excel ZIPs (75219.0) → 1,730 false alarms	ZIP numbers understood; only lost zeros flagged
Agent 3	Quality score always 0 on real files	Weighted share of clean cells (0–100)
Agent 3	271 false outliers on Contents	Log scale for money; skip zero-spread columns
Agent 3	Missing column produced no issue at all	One clear dataset-level issue
Review	One edit value overwrote rows with different values	Per-row editing for mixed groups
Agent 4	Did not exist; audit was a sheet inside the output	Built; single-sheet output + separate audit files
Ingestion	Excel file stayed locked after upload (Windows)	Workbook closed after reading
LLM	Free-tier daily quota → minutes of waiting	Batching, token budget, circuit breaker, key failover
UI	Every decision jumped back to the first tab	Stateful tabs; stay on Review

Demo script and judge questions

A 10-minute walkthrough of the seven stages the brief requires, and answers to likely questions.

Demo (use SOV_K4T9.xlsx)

#	Stage	Where	Show and say
1	Upload & sheet detection	Sidebar → Sheets tab	7 sheets ranked; "Locations" beats "Original from Client A"; header on row 6.
2	Schema detection	Mapping tab	Mapping JSON; "Buildings" → Building Value resolved by values; "Location Name" → City at 0.59 goes to review.
3	Quality report	Data quality tab	Intake score, completeness per field, ≥ 3 issue types: 9999 placeholders, missing addresses, outliers.
4	Recommendations	Review tab	Open one column mapping, one data correction (9999 → blank), one standardisation.
5	Approval workflow	Review tab	Approve All; approve one; reject one with a note → attempt 2 "revised after rejection"; reject again → escalated.
6	Transformation & audit	Export tab	Before/after side by side; finish the queue; Export; show the audit log.
7	Download	Export tab	Download Cleaned_SOV.xlsx: single sheet, 17 headers, no merged cells, blanks not zeros.

Close on the **Overview** tab: the live workflow diagram (bonus), and mention mapping memory. If the network fails, switch off "LLM reasoning" — everything except LLM notes still works.

Questions judges may ask

Q: What if the LLM hallucinates a mapping or a value?

A: It can only choose from the 17 fields or a fixed list of operations; any new value must be an allowed value or appear in the reviewer's note; every answer is re-checked by the same validators as a human edit; and nothing is applied without approval.

Q: Why not let the LLM check the data?

A: Detection must be complete and repeatable on every row — rules do that in seconds and never miss a row. The LLM adds value where judgement is needed: ambiguous headers and re-thinking after a rejection.

Q: How do you avoid overwhelming the reviewer?

A: Grouping (395 identical fixes = 1 item), Approve All for safe format fixes, and no false alarms from spreadsheet quirks. SOV_Q8B3 goes from 3,546 issues to 64 decisions.

Q: How do you know your accuracy isn't overfitted?

A: We built a held-out test with header names not in our vocabulary and kept half of them unseen: 98.3%. We're honest that our team wrote both, so real unseen files are the final test.

Q: What happens with values you can't fix?

A: They are flagged and kept. If they can't be stored in the target type (e.g. "TBD" as money), Agent 4 leaves the cell blank — never 0 — and the original value is kept in the audit log.

Q: How are the agents really separate?

A: Each has its own module, inputs and outputs, and they communicate only through the shared SOVState. Agent 4 cannot run until the review gate is passed.

Q: Is client data safe?

A: API keys are in .env (never in code); identifying values are masked before any LLM call; memory stores only headers; the app runs locally.

Q: Does it work on files it has never seen?

A: Yes — 30 generated files with new layouts and header names were processed with 100% sheet and header detection and 98% mapping; unclear columns go to review instead of being guessed.

How to run it, and where things live

For the team: commands, settings and the project map.

Commands

```
uv sync # install
uv run streamlit run app/ui/streamlit_app.py # web app (recommended)

uv run python -m app.main data/input/SOV_K4T9.xlsx # command line: stops at review
uv run python -m app.main FILE --approve bulk # approve safe items only
uv run python -m app.main FILE --decisions d.json --reviewer alice
uv run python -m app.main FILE --no-llm --no-memory # rules only

uv run pytest # all tests
uv run python -m tests.evaluate_mapping_accuracy # mapping accuracy
uv run python -m tests.evaluate_unseen_variants # held-out test
uv run python -m tests.evaluate_agent3_real # error detection recall
```

Settings (.env)

Setting	Purpose
GROQ_API_KEY, GROQ_API_KEY_2...9 or GROQ_API_KEYS	LLM keys (different Groq accounts → automatic failover)
GROQ_MODEL / AGENT3_LLM_MODEL	Model for Agent 2 / Agent 3 (default openai/gpt-oss-120b)
GROQ_TOKENS_PER_MINUTE	Token budget per key (default 7000)
GROQ_REASONING_EFFORT	Reasoning effort for gpt-oss models (default low)
SOV_MEMORY_PATH	Where mapping memory is stored

Project map

Folder / file	What it is
app/ingestion/	Reading .xlsx / .xls / .csv, file validation
app/agents/sheet_discovery/	Agent 1: header detection, sheet classification, ranking, table extraction
app/agents/schema_mapping/	Agent 2: matchers, keyword evidence, embeddings, LLM client, memory, assignment
app/agents/data_quality/	Agent 3: validators, consolidation, report, normalizers, recommendation engine, policy, review loop, reasoner
app/agents/transformation/	Agent 4: transformation, export, validation, summary
app/pipeline.py, app/main.py	Orchestrator shared by UI and command line
app/ui/	Streamlit web app
app/llm_pool.py, app/llm_rate_limiter.py	API key failover, token budget, quota handling
app/state/sov_state.py	The shared state object
tests/	569 tests and the evaluation scripts
docs/	Step-by-step engineering notes, demo script, compliance matrix

Glossary

Plain-English meanings of the terms used in this guide.

Term	Meaning
SOV (Statement of Values)	A client's spreadsheet listing insured properties and their values.
TIV	Total Insured Value — the sum of building, contents, BI and other values.
BI	Business Interruption / Business Income — money lost while a building can't be used.
Agent	A program with one clear job that reads the shared state and adds its result.
Deterministic	Same input → same output every time; rule-based, no randomness.
LLM	Large Language Model — an AI that reads and writes text (here: gpt-oss-120b on Groq).
Embedding	A list of numbers representing the meaning of a text; similar meanings give similar numbers.
Fuzzy matching	Comparing spellings that are nearly the same ("Constrution" vs "Construction").
Confidence	How sure the system is (0 to 1). Detection confidence = the issue is real; fix confidence = the fix is right.
Recall	Of all real errors, the share we found.
Held-out test	A test on examples deliberately kept away from development, to measure generalisation honestly.
Lossless	A change of form only, not meaning ("No" → "N").
Escalate	Pass a decision to a more senior person; blocks export until resolved.
Audit log	The permanent record of every applied change: before, after, who, when, why.
Rate limit / quota	How many LLM tokens a key may use per minute / per day.
Circuit breaker	Stops calling a failing service for a while instead of retrying endlessly.

This guide reflects the code on branch agent3-pipeline-ui. Numbers were produced by running the system on the provided sample files without the LLM and without memory, so they are reproducible.